Literate Programming

Write it for a human

Francis **Bond**

Computational Linguistics Lab

Palacký University, Olomouc, Czechia

JPC3 — Improving the Czech Wordnet

November 3, 2026

Overview

- The idea: write code for a human to read
- The `.lit.md` format: prose, chunks, diagrams
- **Tangle** the code out; **weave** the document
- A worked example: the Wikidata script
- Your turn

Roadmap

1. The idea
2. The format
3. Tangle and weave
4. The example
5. Your turn

Write It for a Human

- Knuth (1984): a program is a **document** people read
- Explanation first, code as its consequence
- The starter code is cheap; understanding is the bottleneck
- So make the reasoning the main text
- Your report is already a literate layer over your scripts

Roadmap

1. The idea
2. The format
3. Tangle and weave
4. The example
5. Your turn

Faculty of Arts

A `.lit.md` Is Markdown Plus Two Rules

A **named chunk** is defined once; prose explains it first:

````
## Load the CILI map

```python {#load-the-cili-map}
def load_cili() -> dict[str, str]:
    ...
```
````

A **file chunk** is written to disk, and lists the pieces in order:

````
```python {file=wikidata_to_czech.py}
<<script-header>>
<<load-the-cili-map>>
<<query-wikidata>>
```
````

Chunks and Diagrams

- `<<name>>` on its own line expands that chunk
- Indentation is preserved, so Python stays aligned
- A name used twice is joined, so prose can interleave a function
- A ‘‘‘`mermaid` block is a diagram

````
‘‘‘mermaid
flowchart LR
  WD[Wikidata] -->|P5063| ILI --> CS[Czech synset]
‘‘‘
````

Faculty of Arts

Roadmap

1. The idea
2. The format
3. Tangle and weave
4. The example
5. Your turn

Tangle: Code Out

Pull the runnable script out of the document:

```
uv run ../litprog/tangle.py wikidata.lit.md
# -> wikidata_to_czech.py
uv run wikidata_to_czech.py 500
```

- The `.py` is **generated**: edit the `.lit.md`, not it
- Re-tangle after every edit, then run to check
- The tangler is ∼40 lines, read it too

Weave: Document Out

Render the document a human reads:

```
uv run ../litprog/weave.py wikidata.lit.md
# -> wikidata.html  and  wikidata.pdf
```

- Mermaid is rendered by **Kroki** over HTTP: no node, no browser
- Pure Python + `pandoc` + `xelatex`
- GitHub renders the `.lit.md` with the diagram inline anyway

Roadmap

1. The idea
2. The format
3. Tangle and weave
4. The example
5. Your turn

The Wikidata Example

`wikidata/wikidata.lit.md` reads as one story:

- The problem, then a diagram of the pipeline
- An assembly chunk listing the pieces
- One named chunk per piece, prose above each
- A real run at the end: **show the output**

It **tangles** to `wikidata_to_czech.py` (passes `ruff`, runs live) and **weaves** to `wikidata.html` / `.pdf`.

Faculty of Arts

Roadmap

1. The idea
2. The format
3. Tangle and weave
4. The example
5. Your turn

Try It

- Open `wikidata/wikidata.lit.md` and read it top to bottom
- Change a chunk; re-tangle; run: see the `.py` update
- Weave it; open the PDF
- For your own project: one diagram, a few named chunks
- Keep the `.lit.md` the **single source of truth**

The Skill

- `litprog/SKILL.md` teaches Claude Code the conventions
- Copy it to `.claude/skills/literate-programming/`
- Then: “make this snippet literate”, “re-tangle and weave”
- Same rules, so the whole class writes the same way

Thanks

- Write for a human — the machine will cope
- Explanation first, code as its consequence
- Tools and the example: `litprog/` in the JPC3 repo

Now go and make your pipeline readable